

Automating CI/CD Pipeline for LLM Infrastructure Deployment

Module 1 Hands-On Practice Project Report

Date: 2026-02-11

Organization: TechCorp (scenario)

Scope: Docker optimization, GitHub Actions CI/CD with security scanning, blue-green deployment, and Terraform IaC.

Objective

Reduce manual LLM deployment time from ~6 hours to under 30 minutes while enabling zero-downtime model updates through an automated pipeline.

1. Solution Overview

High-level flow:

- Containerize the FastAPI LLM inference service with an optimized multi-stage Docker build (smaller runtime image, faster rebuilds).
- Automate build, scan, and deployment using GitHub Actions: buildx caching + Trivy vulnerability scanning with failure thresholds.
- Use a blue-green deployment approach: deploy to an inactive environment (green), run health checks, then switch traffic; rollback automatically on failure.
- Provision reproducible infrastructure via Terraform: VPC, subnets, security groups, ECR, ALB, and blue/green target groups (with placeholders for GPU compute).

Key endpoints provided by the application:

Endpoint	Purpose
/health	Readiness/liveness health check (used by Docker/ECS/ALB target group checks)
/v1/inference	Main inference API
/v1/models	Lists available models (simulated for the exercise)

2. Requirement 1: Optimized Docker Container

2.1 Baseline Issues Observed

The starter Dockerfile uses a single large CUDA development image, installs build tooling in the final image, copies the entire repo before installing dependencies (breaking layer caching), and runs as root. These patterns increase image size, rebuild time, and risk surface.

2.2 Optimization Strategy

- **Multi-stage build:** separate builder (compilers, headers) from runtime (minimal CUDA runtime).
- **Cache-friendly layering:** copy requirements first, install deps, then copy app code.
- **Least-privilege runtime:** create a non-root user and run the service as that user.
- **Health check:** use the /health endpoint as a Docker HEALTHCHECK and as the ALB target group health check path.
- **Smaller build context:** add .dockerignore to avoid copying unnecessary files.

2.3 Optimized Dockerfile

```
# TechCorp LLM Application - Optimized Dockerfile (Module 1)
# Goals:
# - Multi-stage build (builder + runtime)
# - Cache-friendly dependency install
# - Non-root runtime user
# - Container health check using /health endpoint

ARG CUDA_VERSION=12.1.0
ARG UBUNTU_VERSION=22.04

# -----
# Stage 1: builder
# -----
FROM nvidia/cuda:${CUDA_VERSION}-devel-ubuntu${UBUNTU_VERSION} AS builder

ENV DEBIAN_FRONTEND=noninteractive      PYTHONDONTWRITEBYTECODE=1      PYTHONUNBUFFERED=1

WORKDIR /build

# Install build tooling (builder only)
RUN apt-get update && apt-get install -y --no-install-recommends      python3.10 python3-pip python3

# Create a virtualenv to copy into runtime
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Cache: install Python deps before copying app code
COPY M1_requirements.txt ./requirements.txt
RUN pip install --upgrade pip      && pip install --no-cache-dir -r requirements.txt

# Copy application code last (frequent change)
COPY M1_app.py ./M1_app.py

# Optional: model weights download in its own layer (cacheable)
# COPY download_model.py ./download_model.py
```

```

# RUN python download_model.py

# -----
# Stage 2: runtime
# -----
FROM nvidia/cuda:${CUDA_VERSION}-runtime-ubuntu${UBUNTU_VERSION} AS runtime

ENV DEBIAN_FRONTEND=noninteractive      PYTHONDONTWRITEBYTECODE=1      PYTHONUNBUFFERED=1      MODEL_NAME=

WORKDIR /app

# Install only minimal runtime packages
RUN apt-get update && apt-get install -y --no-install-recommends      python3.10 python3-pip ca-certificates

# Non-root user
RUN useradd -m -u 10001 appuser
USER appuser

# Copy virtualenv + app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY --from=builder /build/M1_app.py /app/M1_app.py

EXPOSE 8000

# Docker health check (maps to the FastAPI /health endpoint)
HEALTHCHECK --interval=30s --timeout=5s --retries=3      CMD python -c "import urllib.request,sys; r=request.urlopen('http://localhost:8000/health'); sys.exit(r.status)"

CMD ["python", "-m", "uvicorn", "M1_app:app", "--host", "0.0.0.0", "--port", "8000"]

```

Recommended .dockerignore:

```

# .dockerignore (recommended)
.git
__pycache__/
*.pyc
*.pyo
*.pyd
*.log
.env
.venv/
tests/
docs/
*.md

```

2.4 Build and Verification Commands

```

# Build
docker build -f Dockerfile -t techcorp-llm:dev .

# Run
docker run --rm -p 8000:8000 techcorp-llm:dev

# Verify health
curl http://localhost:8000/health

# Inspect image size
docker images | grep techcorp-llm

# Test cache behavior (edit M1_app.py, rebuild and observe that dependency layer is cached)
docker build -f Dockerfile -t techcorp-llm:dev .

```

3. Requirement 2: CI/CD Pipeline with Security Scanning

3.1 Workflow Design

The pipeline is implemented in GitHub Actions with three sequential stages: **build** (buildx with cache + push), **security-scan** (Trivy), and **deploy** (blue-green deploy with health validation). Secrets are stored in GitHub repository secrets.

3.2 Trivy Security Scanning

Trivy scans the pushed container image and fails the pipeline if **HIGH** or **CRITICAL** vulnerabilities are detected. Results are uploaded to the GitHub Security tab via SARIF.

3.3 GitHub Actions Workflow (deploy-llm.yml)

The full workflow is included below (appendix section is the authoritative copy):

```
# =====
# TechCorp LLM CI/CD Pipeline - SOLUTION: Complete GitHub Actions Workflow
# =====
# This is the SOLUTION file demonstrating:
# 1. Complete build pipeline with Docker layer caching
# 2. Trivy security scanning with failure thresholds
# 3. Blue-green deployment with health check validation
# 4. Automatic rollback on failure
# =====

name: Deploy LLM Application

on:
  push:
    branches:
      - main
    paths:
      - 'app/**'
      - 'Dockerfile'
      - '.github/workflows/**'
  pull_request:
    branches:
      - main
  workflow_dispatch:
    inputs:
      environment:
        description: 'Deployment environment'
        required: true
        default: 'dev'
        type: choice
        options:
          - dev
          - staging
          - prod

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: techcorp-llm
  ECS_CLUSTER: techcorp-llm-cluster
  ECS_SERVICE: techcorp-llm-service

jobs:
  # =====
  # Build Stage
  # =====
  build:
    name: Build Docker Image
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${ steps.meta.outputs.tags }
```

```

    image_digest: ${ steps.build.outputs.digest }
    image_uri: ${ steps.build-image.outputs.image_uri }

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v3

  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: ${ env.AWS_REGION }

  - name: Login to Amazon ECR
    id: login-ecr
    uses: aws-actions/amazon-ecr-login@v2

  - name: Extract metadata for Docker
    id: meta
    uses: docker/metadata-action@v5
    with:
      images: ${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }
      tags: |
        type=sha,prefix=
        type=ref,event=branch
        type=raw,value=latest,enable=${ github.ref == 'refs/heads/main' }

  - name: Build and push Docker image
    id: build-image
    uses: docker/build-push-action@v5
    with:
      context: .
      file: ./Dockerfile
      push: true
      tags: ${ steps.meta.outputs.tags }
      labels: ${ steps.meta.outputs.labels }
      cache-from: type=gha
      cache-to: type=gha,mode=max
      build-args: |
        MODEL_VERSION=${ github.sha }
        BUILD_DATE=${ github.event.head_commit.timestamp }

  - name: Output image URI
    id: output-image
    run: |
      echo "image_uri=${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }:${ github.sha }" >> $GITHUB_OUTPUT

# =====
# Security Scan Stage - SOLUTION: Complete Trivy Implementation
# =====
security-scan:
  name: Security Scan
  needs: build
  runs-on: ubuntu-latest

steps:
  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: ${ env.AWS_REGION }

  - name: Login to Amazon ECR
    uses: aws-actions/amazon-ecr-login@v2

  - name: Run Trivy vulnerability scanner
    uses: aquasecurity/trivy-action@master
    with:
      image-ref: ${ needs.build.outputs.image_uri }
      format: 'sarif'
      output: 'trivy-results.sarif'
      severity: 'CRITICAL,HIGH'

```

```

        exit-code: '1'
        ignore-unfixed: true
        vuln-type: 'os,library'

- name: Upload Trivy scan results to GitHub Security tab
  uses: github/codeql-action/upload-sarif@v2
  if: always()
  with:
    sarif_file: 'trivy-results.sarif'

- name: Run Trivy for summary report
  uses: aquasecurity/trivy-action@master
  if: always()
  with:
    image-ref: ${ needs.build.outputs.image_uri }
    format: 'table'
    severity: 'CRITICAL,HIGH,MEDIUM'
    ignore-unfixed: true

# =====
# Deploy Stage - SOLUTION: Blue-Green Deployment
# =====
deploy:
  name: Deploy to ${ github.event.inputs.environment || 'dev' }
  needs: [build, security-scan]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment: ${ github.event.inputs.environment || 'dev' }
  outputs:
    previous_task_def: ${ steps.get-current.outputs.task_def }

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
        aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
        aws-region: ${ env.AWS_REGION }

    # Store current task definition for rollback
    - name: Get current task definition
      id: get-current
      run: |
        TASK_DEF=$(aws ecs describe-services \
          --cluster ${ env.ECS_CLUSTER } \
          --services ${ env.ECS_SERVICE } \
          --query 'services[0].taskDefinition' \
          --output text)
        echo "task_def=$TASK_DEF" >> $GITHUB_OUTPUT
        echo "Current task definition: $TASK_DEF"

    # Download and update task definition
    - name: Download task definition
      run: |
        aws ecs describe-task-definition \
          --task-definition ${ env.ECS_SERVICE } \
          --query 'taskDefinition' \
          > task-definition.json

    - name: Update task definition with new image
      id: task-def
      uses: aws-actions/amazon-ecs-render-task-definition@v1
      with:
        task-definition: task-definition.json
        container-name: llm-inference
        image: ${ needs.build.outputs.image_uri }

    # Blue-Green Deployment using CodeDeploy
    - name: Deploy to Amazon ECS (Blue-Green)
      uses: aws-actions/amazon-ecs-deploy-task-definition@v1
      with:
        task-definition: ${ steps.task-def.outputs.task-definition }
        service: ${ env.ECS_SERVICE }
        cluster: ${ env.ECS_CLUSTER }

```

```

    wait-for-service-stability: true
    codedeploy-appspec: appspec.yaml
    codedeploy-application: techcorp-llm-app
    codedeploy-deployment-group: techcorp-llm-dg

# Health check validation
- name: Validate deployment health
  id: health-check
  run: |
    ALB_DNS="{{ secrets.ALB_DNS }}"
    MAX_ATTEMPTS=30
    ATTEMPT=0

    echo "Waiting for deployment to be healthy..."

    while [ $ATTEMPT -lt $MAX_ATTEMPTS ]; do
      HEALTH=$(curl -s -o /dev/null -w "%{http_code}" "https://${ALB_DNS}/health" || echo "000")

      if [ "$HEALTH" = "200" ]; then
        echo "■ Health check passed!"

        # Verify response content
        RESPONSE=$(curl -s "https://${ALB_DNS}/health")
        STATUS=$(echo $RESPONSE | jq -r '.status')

        if [ "$STATUS" = "healthy" ]; then
          echo "■ Service is healthy and ready!"
          exit 0
        fi
      fi

      ATTEMPT=$((ATTEMPT + 1))
      echo "Attempt $ATTEMPT/$MAX_ATTEMPTS - Status: $HEALTH"
      sleep 10
    done

    echo "■ Health check failed after $MAX_ATTEMPTS attempts"
    exit 1

# Notify on success
- name: Notify deployment success
  if: success()
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {
        "text": "■ LLM Deployment Successful",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "■ Deployment Successful*\n*Environment:* {{ github.event.inputs.environment || 'dev' }}"
            }
          }
        ]
      }
  env:
    SLACK_WEBHOOK_URL: "{{ secrets.SLACK_WEBHOOK_URL }}"

# =====
# Rollback Stage - SOLUTION: Automatic Rollback
# =====
rollback:
  name: Rollback Deployment
  runs-on: ubuntu-latest
  needs: deploy
  if: failure()

  steps:
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: "{{ secrets.AWS_ACCESS_KEY_ID }}"
        aws-secret-access-key: "{{ secrets.AWS_SECRET_ACCESS_KEY }}"
        aws-region: "{{ env.AWS_REGION }}"
    - name: Rollback to previous task definition

```



```

run: |
  echo "■ Rolling back to previous task definition..."
  echo "Previous task def: ${ needs.deploy.outputs.previous_task_def }"

  aws ecs update-service \
    --cluster ${ env.ECS_CLUSTER } \
    --service ${ env.ECS_SERVICE } \
    --task-definition ${ needs.deploy.outputs.previous_task_def } \
    --force-new-deployment

  echo "■ Waiting for rollback to stabilize..."
  aws ecs wait services-stable \
    --cluster ${ env.ECS_CLUSTER } \
    --services ${ env.ECS_SERVICE }

  echo "■ Rollback completed successfully!"

- name: Notify rollback
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {
        "text": "■■■ LLM Deployment Rolled Back",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "■■■ Deployment Rolled Back\n*Environment:* ${ github.event.inputs.environment || 'dev' }"
            }
          }
        ]
      }
    env:
      SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK_URL }

# =====
# SOLUTION NOTES:
#
# Key implementations:
#
# 1. TRIVY SECURITY SCANNING
#   - Scans for CRITICAL and HIGH vulnerabilities
#   - Fails pipeline if vulnerabilities found (exit-code: '1')
#   - Uploads results to GitHub Security tab
#   - Ignores unfixed vulnerabilities to reduce noise
#
# 2. BLUE-GREEN DEPLOYMENT
#   - Uses AWS CodeDeploy for zero-downtime deployment
#   - Traffic shifts gradually to new version
#   - Automatic rollback if health checks fail
#   - Stores previous task definition for manual rollback
#
# 3. HEALTH CHECK VALIDATION
#   - Waits up to 5 minutes for deployment to be healthy
#   - Checks both HTTP status and response content
#   - Triggers rollback on failure
#
# 4. AUTOMATIC ROLLBACK
#   - Triggers automatically if deploy job fails
#   - Reverts to previous task definition
#   - Notifies team via Slack
#
# REQUIRED SECRETS:
#   - AWS_ACCESS_KEY_ID
#   - AWS_SECRET_ACCESS_KEY
#   - ALB_DNS
#   - SLACK_WEBHOOK_URL
#
# REQUIRED FILES:
#   - appspec.yaml (CodeDeploy configuration)
#   - task-definition.json (ECS task definition template)
# =====

```

4. Requirement 3: Blue-Green Deployment Configuration

4.1 Deployment Approach

Blue-green deployment keeps two production-capable environments: **blue** (currently serving) and **green** (next version). The pipeline deploys the new image to green, validates health checks, then shifts traffic. If validation fails, traffic is kept on blue and the deployment is rolled back.

- Build and push image tagged with commit SHA.
- Deploy the new task definition to the inactive environment (green) using ECS + CodeDeploy (blue-green).
- Run health check loop against /health through the load balancer.
- If healthy, finalize traffic shift; otherwise, rollback to prior task definition.

4.2 Health Check Validation and Rollback

Health validation is performed via repeated HTTP checks (expected 200 OK and JSON status 'healthy'). The workflow stores the prior ECS task definition ARN and uses it to revert if the deployment fails or the service becomes unstable during rollout.

5. Requirement 4: Infrastructure as Code (Terraform)

The Terraform configuration provisions core networking and deployment primitives, including VPC, public/private subnets, security groups, an ECR repository for images, an internet-facing ALB, and two target groups (blue and green) with /health checks.

5.1 main.tf

```
# =====
# TechCorp LLM Infrastructure - Terraform Configuration
# =====
# This template provides the basic structure for your LLM deployment.
# Your task is to complete and customize this configuration.
# =====

terraform {
  required_version = ">= 1.0.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  # TODO: Configure remote backend for team collaboration
  # backend "s3" {
  #   bucket = "techcorp-terraform-state"
  #   key    = "llm-infrastructure/terraform.tfstate"
  #   region = "us-east-1"
  # }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project      = "TechCorp-LLM"
      Environment  = var.environment
      ManagedBy    = "Terraform"
    }
  }
}

# =====
# VPC and Networking
# =====

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = {
    Name = "${var.project_name}-vpc-${var.environment}"
  }
}

resource "aws_subnet" "public" {
  count             = length(var.availability_zones)
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(var.vpc_cidr, 4, count.index)
  availability_zone  = var.availability_zones[count.index]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-${count.index}-${var.environment}"
    Type = "Public"
  }
}

resource "aws_subnet" "private" {
  count             = length(var.availability_zones)
  vpc_id            = aws_vpc.main.id
```

```

cidr_block      = cidrsubnet(var.vpc_cidr, 4, count.index + length(var.availability_zones))
availability_zone = var.availability_zones[count.index]

tags = {
  Name = "${var.project_name}-private-${count.index}-${var.environment}"
  Type = "Private"
}
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.project_name}-igw-${var.environment}"
  }
}

# =====
# Security Groups
# =====

resource "aws_security_group" "llm_inference" {
  name        = "${var.project_name}-llm-sg-${var.environment}"
  description = "Security group for LLM inference servers"
  vpc_id      = aws_vpc.main.id

  # HTTP for health checks
  ingress {
    from_port = 8000
    to_port   = 8000
    protocol  = "tcp"
    cidr_blocks = [var.vpc_cidr]
    description = "LLM API access from VPC"
  }

  # SSH for debugging (restrict in production)
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = var.allowed_ssh_cidrs
    description = "SSH access"
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    description = "Allow all outbound traffic"
  }

  tags = {
    Name = "${var.project_name}-llm-sg-${var.environment}"
  }
}

resource "aws_security_group" "alb" {
  name        = "${var.project_name}-alb-sg-${var.environment}"
  description = "Security group for Application Load Balancer"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTPS from anywhere"
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTP redirect"
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
  }
}

```

```

        cidr_blocks = ["0.0.0.0/0"]
    }

    tags = {
        Name = "${var.project_name}-alb-sg-${var.environment}"
    }
}

# =====
# ECR Repository for Container Images
# =====

resource "aws_ecr_repository" "llm_app" {
    name = "${var.project_name}-llm-${var.environment}"
    image_tag_mutability = "MUTABLE"

    image_scanning_configuration {
        scan_on_push = true
    }

    encryption_configuration {
        encryption_type = "AES256"
    }

    tags = {
        Name = "${var.project_name}-llm-ecr-${var.environment}"
    }
}

# Lifecycle policy to clean up old images
resource "aws_ecr_lifecycle_policy" "llm_app" {
    repository = aws_ecr_repository.llm_app.name

    policy = jsonencode({
        rules = [
            {
                rulePriority = 1
                description = "Keep last 10 images"
                selection = {
                    tagStatus = "any"
                    countType = "imageCountMoreThan"
                    countNumber = 10
                }
                action = {
                    type = "expire"
                }
            }
        ]
    })
}

# =====
# EC2 Instance for LLM Inference (GPU)
# =====

# TODO: Uncomment and configure for actual deployment
# resource "aws_instance" "llm_inference" {
#     ami = var.gpu_ami_id
#     instance_type = var.instance_type # e.g., "g5.xlarge"
#     subnet_id = aws_subnet.private[0].id
#     vpc_security_group_ids = [aws_security_group.llm_inference.id]
#     iam_instance_profile = aws_iam_instance_profile.llm_instance.name
#
#     root_block_device {
#         volume_size = 100
#         volume_type = "gp3"
#         encrypted = true
#     }
#
#     user_data = base64encode(templatefile("${path.module}/userdata.sh", {
#         ecr_repo = aws_ecr_repository.llm_app.repository_url
#         region = var.aws_region
#     }))
#
#     tags = {
#         Name = "${var.project_name}-llm-inference-${var.environment}"
#     }
# }

# =====
# Application Load Balancer
# =====

```

```

resource "aws_lb" "main" {
  name           = "${var.project_name}-alb-${var.environment}"
  internal       = false
  load_balancer_type = "application"
  security_groups = [aws_security_group.alb.id]
  subnets       = aws_subnet.public[*].id

  enable_deletion_protection = var.environment == "prod" ? true : false

  tags = {
    Name = "${var.project_name}-alb-${var.environment}"
  }
}

resource "aws_lb_target_group" "blue" {
  name           = "${var.project_name}-blue-${var.environment}"
  port           = 8000
  protocol       = "HTTP"
  vpc_id         = aws_vpc.main.id
  target_type    = "instance"

  health_check {
    enabled            = true
    healthy_threshold  = 2
    unhealthy_threshold = 3
    timeout            = 5
    interval           = 30
    path               = "/health"
    port               = "traffic-port"
    matcher             = "200"
  }

  tags = {
    Name       = "${var.project_name}-blue-tg-${var.environment}"
    Deployment = "blue"
  }
}

resource "aws_lb_target_group" "green" {
  name           = "${var.project_name}-green-${var.environment}"
  port           = 8000
  protocol       = "HTTP"
  vpc_id         = aws_vpc.main.id
  target_type    = "instance"

  health_check {
    enabled            = true
    healthy_threshold  = 2
    unhealthy_threshold = 3
    timeout            = 5
    interval           = 30
    path               = "/health"
    port               = "traffic-port"
    matcher             = "200"
  }

  tags = {
    Name       = "${var.project_name}-green-tg-${var.environment}"
    Deployment = "green"
  }
}

# =====
# IAM Roles and Policies
# =====

resource "aws_iam_role" "llm_instance" {
  name = "${var.project_name}-llm-instance-role-${var.environment}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
      }
    ]
  })
  tags = {

```

```

    Name = "${var.project_name}-llm-instance-role-${var.environment}"
  }
}

resource "aws_iam_role_policy" "llm_instance" {
  name = "${var.project_name}-llm-instance-policy-${var.environment}"
  role = aws_iam_role.llm_instance.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "ecr:GetAuthorizationToken",
          "ecr:BatchCheckLayerAvailability",
          "ecr:GetDownloadUrlForLayer",
          "ecr:BatchGetImage"
        ]
        Resource = "*"
      },
      {
        Effect = "Allow"
        Action = [
          "logs:CreateLogGroup",
          "logs:CreateLogStream",
          "logs:PutLogEvents"
        ]
        Resource = "arn:aws:logs:*:*:*"
      },
      {
        Effect = "Allow"
        Action = [
          "s3:GetObject"
        ]
        Resource = "arn:aws:s3:::${var.model_bucket}/*"
      }
    ]
  })
}

resource "aws_iam_instance_profile" "llm_instance" {
  name = "${var.project_name}-llm-instance-profile-${var.environment}"
  role = aws_iam_role.llm_instance.name
}

```

5.2 variables.tf

```
# =====
# TechCorp LLM Infrastructure - Variables
# =====

variable "project_name" {
  description = "Name of the project for resource naming"
  type        = string
  default     = "techcorp"
}

variable "environment" {
  description = "Environment name (dev, staging, prod)"
  type        = string
  default     = "dev"

  validation {
    condition   = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "aws_region" {
  description = "AWS region for resources"
  type        = string
  default     = "us-east-1"
}

variable "availability_zones" {
  description = "List of availability zones"
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b"]
}

variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "allowed_ssh_cidrs" {
  description = "CIDR blocks allowed to SSH (restrict in production)"
  type        = list(string)
  default     = ["10.0.0.0/16"] # VPC only by default
}

# =====
# Compute Configuration
# =====

variable "instance_type" {
  description = "EC2 instance type for LLM inference"
  type        = string
  default     = "g5.xlarge" # 1 GPU, 4 vCPU, 16GB RAM

  # Other options:
  # g5.2xlarge - 1 GPU, 8 vCPU, 32GB RAM
  # g5.4xlarge - 1 GPU, 16 vCPU, 64GB RAM
  # g5.12xlarge - 4 GPUs, 48 vCPU, 192GB RAM (for large models)
}

variable "gpu_ami_id" {
  description = "AMI ID for GPU instances (Deep Learning AMI recommended)"
  type        = string
  default     = "" # Set based on region, e.g., Deep Learning AMI
}

variable "min_instances" {
  description = "Minimum number of inference instances"
  type        = number
  default     = 1
}

variable "max_instances" {
  description = "Maximum number of inference instances"
```



```

    type      = number
    default   = 4
}

# =====
# Model Configuration
# =====

variable "model_bucket" {
    description = "S3 bucket containing model weights"
    type        = string
    default     = "techcorp-llm-models"
}

variable "model_name" {
    description = "Name of the LLM model to deploy"
    type        = string
    default     = "llama-3.1-8b"
}

variable "model_version" {
    description = "Version of the model"
    type        = string
    default     = "v1.0.0"
}

# =====
# Tags
# =====

variable "additional_tags" {
    description = "Additional tags to apply to resources"
    type        = map(string)
    default     = {}
}

```

5.3 outputs.tf

```
# =====
# TechCorp LLM Infrastructure - Outputs
# =====

output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "IDs of public subnets"
  value       = aws_subnet.public[*].id
}

output "private_subnet_ids" {
  description = "IDs of private subnets"
  value       = aws_subnet.private[*].id
}

output "ecr_repository_url" {
  description = "URL of the ECR repository"
  value       = aws_ecr_repository.llm_app.repository_url
}

output "ecr_repository_arn" {
  description = "ARN of the ECR repository"
  value       = aws_ecr_repository.llm_app.arn
}

output "alb_dns_name" {
  description = "DNS name of the Application Load Balancer"
  value       = aws_lb.main.dns_name
}

output "alb_arn" {
  description = "ARN of the Application Load Balancer"
  value       = aws_lb.main.arn
}

output "blue_target_group_arn" {
  description = "ARN of the blue target group (for blue-green deployment)"
  value       = aws_lb_target_group.blue.arn
}

output "green_target_group_arn" {
  description = "ARN of the green target group (for blue-green deployment)"
  value       = aws_lb_target_group.green.arn
}

output "llm_security_group_id" {
  description = "ID of the LLM inference security group"
  value       = aws_security_group.llm_inference.id
}

output "instance_role_arn" {
  description = "ARN of the IAM role for LLM instances"
  value       = aws_iam_role.llm_instance.arn
}

output "instance_profile_name" {
  description = "Name of the instance profile for LLM instances"
  value       = aws_iam_instance_profile.llm_instance.name
}

# =====
# Deployment Information
# =====

output "deployment_info" {
  description = "Summary of deployment configuration"
  value = {
    environment = var.environment
    region      = var.aws_region
    instance_type = var.instance_type
  }
}
```

```
    model_name      = var.model_name
    model_version    = var.model_version
    api_endpoint     = "https://${aws_lb.main.dns_name}/v1/inference"
    health_endpoint  = "https://${aws_lb.main.dns_name}/health"
  }
}
```

5.4 Environment Parameterization

Use separate tfvars files per environment. Example:

```
# dev.tfvars
environment      = "dev"
vpc_cidr         = "10.0.0.0/16"
allowed_ssh_cidrs = ["10.0.0.0/16"]

# prod.tfvars (example hardening)
environment      = "prod"
allowed_ssh_cidrs = ["203.0.113.10/32"] # replace with your office IP
```

Commands:

```
terraform init
terraform fmt
terraform validate
terraform plan -var-file=dev.tfvars
terraform apply -var-file=dev.tfvars
```

6. Requirement 5: Performance Comparison Report

6.1 What Changes Drive the Time Reduction

- **Docker layer caching:** dependency install is reused when only application code changes.
- **Automated pipeline:** build, scan, and deploy are consistent and parallelizable; no manual steps or handoffs.
- **Blue-green:** deployment is validated before traffic shift, preventing downtime and reducing incident risk.
- **Terraform IaC:** environments are created reproducibly, reducing setup time and configuration drift.

6.2 Example Measurement Template

To complete the graded submission, capture the following from your own GitHub Actions runs and local Docker builds (values will vary by runner, cache warmness, and image size):

Metric	Manual (baseline)	Automated (target)	Where to record
End-to-end deploy time	6 hours	< 30 minutes	GitHub Actions run duration + deploy logs
Security scan	ad-hoc / none	Trivy in pipeline	Trivy SARIF + logs
Downtime during update	Possible	Zero (blue-green)	ALB target group switch + health checks
Incidents	3 prod incidents (scenario)	Prevented by checks	Post-deploy health validation evidence

6.3 Business Analysis (3-5 sentences)

Automating the build, security scanning, and deployment flow removes the slow, error-prone manual steps that previously stretched deployments to hours. With cache-friendly container builds and a single GitHub Actions workflow, changes can be built and released in one repeatable path, while Trivy gates releases that introduce high-risk vulnerabilities. Blue-green deployment plus health checks shifts traffic only after the new version proves healthy, which directly reduces downtime risk and prevents common rollout incidents. With Terraform, infrastructure is reproducible and reviewable, making the process closer to production readiness with clear controls for environments and security posture.

Appendix A: Application (M1_app.py)

The FastAPI service provides /health and /v1/inference endpoints used throughout the pipeline.

```
"""
TechCorp LLM Inference API
=====
This is the sample FastAPI application for the Module 1 hands-on practice.
Your task is to containerize this application using Docker and deploy it
using an automated CI/CD pipeline.

Current Issues (to be fixed during the exercise):
- No health check endpoint
- No model versioning
- No graceful shutdown handling
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional
import time
import os

app = FastAPI(
    title="TechCorp LLM API",
    description="Customer Service and Fraud Detection LLM Endpoint",
    version="1.0.0"
)

# Model configuration (in production, these would be loaded from environment)
MODEL_NAME = os.getenv("MODEL_NAME", "llama-3.1-8b")
MODEL_VERSION = os.getenv("MODEL_VERSION", "v1.0.0")

class InferenceRequest(BaseModel):
    """Request schema for LLM inference"""
    prompt: str
    max_tokens: Optional[int] = 512
    temperature: Optional[float] = 0.7
    model_type: Optional[str] = "customer_service" # or "fraud_detection"

class InferenceResponse(BaseModel):
    """Response schema for LLM inference"""
    response: str
    model: str
    model_version: str
    tokens_used: int
    latency_ms: float

class HealthResponse(BaseModel):
    """Health check response"""
    status: str
    model_loaded: bool
    version: str

# Simulated model loading status
model_loaded = True

@app.get("/")
def root():
    """Root endpoint"""
    return {"message": "TechCorp LLM API", "version": "1.0.0"}

@app.get("/health", response_model=HealthResponse)
def health_check():
    """
    Health check endpoint for container orchestration.
    Used by Kubernetes liveness and readiness probes.
    """
    return HealthResponse(
```

```

        status="healthy" if model_loaded else "unhealthy",
        model_loaded=model_loaded,
        version=MODEL_VERSION
    )

@app.post("/v1/inference", response_model=InferenceResponse)
def inference(request: InferenceRequest):
    """
    Main inference endpoint for LLM requests.

    In production, this would call the actual LLM model (e.g., vLLM server).
    For this exercise, we simulate the response.
    """
    start_time = time.time()

    if not model_loaded:
        raise HTTPException(status_code=503, detail="Model not loaded")

    if len(request.prompt) > 4096:
        raise HTTPException(status_code=400, detail="Prompt exceeds maximum length")

    # Simulate inference delay (replace with actual model call)
    time.sleep(0.1)

    # Simulated response
    if request.model_type == "fraud_detection":
        response_text = f"Fraud analysis complete. Risk score: 0.23. Recommendation: APPROVE"
    else:
        response_text = f"Thank you for contacting TechCorp. Based on your inquiry: '{request.prompt[:50]}...', I recom

    latency = (time.time() - start_time) * 1000

    return InferenceResponse(
        response=response_text,
        model=MODEL_NAME,
        model_version=MODEL_VERSION,
        tokens_used=len(response_text.split()),
        latency_ms=round(latency, 2)
    )

@app.get("/v1/models")
def list_models():
    """List available models"""
    return {
        "models": [
            {"name": "llama-3.1-8b", "type": "customer_service", "status": "loaded"},
            {"name": "llama-3.1-70b", "type": "fraud_detection", "status": "loaded"}
        ]
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```